

Heverton Eduardo Peres

60+ Providers: Roteamento Inteligente de Modelos & Time-Traveling Stream Rules: Course-Correction Inteligente

Obra técnica de literatura especializada, produzida e diagramada conforme as normas ABNT para publicação editorial.

Brasil
2026

Sumário

60+ Providers: Roteamento Inteligente de Modelos	3
Uma frota de motores para cada tarefa	3
O Universo dos Providers	3
Frontier APIs	3
Coding Plans	4
Run-it-yourself	4
Por Que Ter Mais de Um Provider?	4
Os 10 Roles de Modelo	4
Configurando seu Primeiro Provider	5
Adicionando Ollama: Self-Hosting Gratuito	6
Montando uma Fallback Chain	7
Path-Scoped Models	7
Round-Robin de Credenciais	8
Os 4 Knobs de Roteamento	8
Cena de contraste: o navio que ficou parado	9
Armadilhas comuns	9
Próximos Passos	9
Time-Traveling Stream Rules: Course-Correction Inteligente	11
GPS que corrige a rota em tempo real	11
O Problema: Modelos que Saem do Script	11
A Solução: Stream Rules — Regras que Dormem até Serem Necessárias	12
Dois Modos de Correção	12
A metáfora do navio em oceano	13
Estrutura de uma Stream Rule	13
Regras com Múltiplos Patterns	14
Regras com Escopo de Path	14
Monitoramento em Tempo Real	15
Regras Persistentes vs. Regras de Sessão	15
Armadilhas comuns	15
Próximos Passos	16

60+ Providers: Roteamento Inteligente de Modelos

Uma frota de motores para cada tarefa

No capítulo anterior, você dominou a browser tool e a computer tool — o estaleiro se estendeu por toda a superfície do oceano digital, de portos (websites) ao convés do desktop.

Agora vamos mais fundo: e se cada tarefa pudesse ser executada pelo motor certo do navio? Imagine que seu estaleiro tivesse acesso a dezenas de motores diferentes — cada um com uma potência, um custo e uma eficiência distintos. Tarefa leve? Motor econômico. Raciocínio profundo? Motor pesado. Análise de imagem? Motor especializado.

É exatamente isso que o OMP oferece com seus mais de 60 providers de LLM.

O Universo dos Providers

Um provider é simplesmente uma fonte de modelos de linguagem. Cada provider oferece um ou mais modelos, cada um com suas características — custo por token, velocidade de resposta, janela de contexto e qualidade de raciocínio.

O OMP categoriza os providers em três grandes grupos.

Frontier APIs

As grandes plataformas que hospedam modelos de ponta. OpenAI (GPT-4o, GPT-4.1, o3-mini, o4-mini), Anthropic (Claude Opus 4, Claude Sonnet 4, Claude Haiku), Google (Gemini 2.5 Pro, Gemini 2.5 Flash), xAI (Grok 4, Grok 4 Fast), DeepSeek (DeepSeek R1, DeepSeek V3), e Mistral (Mistral Large, Codestral).

Essas APIs são acessadas via chave de API — você paga por token consumido. É como alugar um motor de um estaleiro externo: você usa quando precisa, paga pelo tempo de uso e devolve quando termina.

Coding Plans

Assinaturas que incluem acesso a modelos. OpenAI Pro, Anthropic Max, Google AI Ultra. São como ter um contrato de manutenção com o estaleiro: você paga uma taxa fixa e tem acesso garantido a qualquer motor da frota.

Run-it-yourself

Modelos que rodam na sua própria infraestrutura. Ollama (modelos locais como Llama, Mistral, Qwen sem custo por token), vLLM (servidor de inferência para equipes com GPUs dedicadas), e LiteLLM (proxy unificado que converte qualquer provider em formato OpenAI).

É como construir seu próprio motor no estaleiro: custo inicial maior, mas controle total e custo marginal zero. Para um iniciante, Ollama é a porta de entrada — basta instalar, baixar um modelo e apontar o OMP para `localhost:11434`.

Por Que Ter Mais de Um Provider?

Cada modelo tem pontos fortes e fracos. O Claude Opus 4 é excepcional em raciocínio profundo, mas custa mais. O Gemini Flash é barato e rápido, mas menos preciso em tarefas complexas. O GPT-4o equilibra custo e qualidade. Ter apenas um provider é como ter um navio com um único motor: se ele quebra ou fica caro demais, toda a operação para.

O OMP resolve isso com quatro mecanismos de roteamento: Model Roles, Fallback Chains, Path-Scoped Models e Round-Robin Credentials.

Os 10 Roles de Modelo

O OMP define 10 funções que cada modelo pode assumir. Cada role mapeia para um modelo diferente no seu `models.yml`.

Role	Função	Exemplo de Uso
<code>default</code>	Turnos normais de conversa	Editar código, responder perguntas
<code>smol</code>	Fan-out barato de subagentes	Tarefas paralelas simples
<code>slow</code>	Raciocínio profundo	Arquitetura complexa, debug difícil
<code>plan</code>	Modo plano (planejamento)	Criar roadmaps, analisar arquitetura
<code>commit</code>	Geração de changelogs	Mensagens de commit, PR descriptions
<code>vision</code>	Análise de imagens	Screenshots, mockups, diagramas
<code>designer</code>	Design de interfaces	Layout, componentes, CSS
<code>task</code>	Orquestração de tarefas	Gerenciar subagentes, coordenar
<code>advisor</code>	Revisor inline	Segundo olho em cada turno
<code>tiny</code>	Tarefas leves	Validação, formatação, lint

Esses roles são a inteligência do roteamento. Quando você define `smol: openai/gpt-4o-mini`, está dizendo: “tarefas baratas e paralelas vão para o modelo mais econômico”. Quando define `slow: anthropic/claude-opus-4-0`, está dizendo: “raciocínio profundo vai para o modelo mais inteligente”.

Configurando seu Primeiro Provider

Vamos começar pelo básico: configurar um provider frontier. O arquivo de configuração de modelos do OMP fica em `~/.omp/agent/models.yml`.

```
# ~/.omp/agent/models.yml

providers:
  openai:
    apiKey: "<sua-chave-openai>"
    api: openai-completions
    models:
      - id: gpt-4o
        name: GPT-4o
        contextWindow: 128000
        maxTokens: 16384
      - id: gpt-4o-mini
        name: GPT-4o Mini
```

```

contextWindow: 128000
maxTokens: 16384

anthropic:
  apiKey: "<sua-chave-anthropic>"
  api: anthropic-messages
  models:
    - id: claude-sonnet-4-20250514
      name: Claude Sonnet 4
      contextWindow: 200000
      maxTokens: 16000

modelRoles:
  default: openai/gpt-4o
  smol: openai/gpt-4o-mini
  slow: anthropic/claude-sonnet-4-20250514

```

Com essa configuração, o OMP já roda com três modelos diferentes. Quando você conversa normalmente, usa o GPT-4o (role `default`). Quando o OMP spawna subagentes para tarefas simples, usa o GPT-4o Mini (role `smol`). Quando precisa de raciocínio profundo, usa o Claude Sonnet 4 (role `slow`).

Adicionando Ollama: Self-Hosting Gratuito

Para rodar modelos locais sem custo por token, instale o Ollama e configure-o no OMP.

```

providers:
  ollama:
    baseUrl: http://localhost:11434/v1
    api: openai-completions
    apiKey: dummy
    models:
      - id: llama3.1
        name: Llama 3.1 8B
        contextWindow: 128000
        maxTokens: 4096
      - id: qwen2.5-coder
        name: Qwen 2.5 Coder 7B
        contextWindow: 32000
        maxTokens: 4096

modelRoles:
  default: ollama/llama3.1

```

```
smol: ollama/qwen2.5-coder
tiny: ollama/qwen2.5-coder
```

Agora seu estaleiro tem um motor local (Ollama) e pode funcionar mesmo sem internet. Para tarefas que exigem mais potência, você mantém os providers frontier como alternativa.

Montando uma Fallback Chain

O verdadeiro poder do roteamento emerge quando você configura fallback chains. Imagine que você depende do Claude Sonnet 4 para tarefas importantes, mas ele pode ficar indisponível ou atingir rate limit. A fallback chain garante que outra opção entre em ação.

```
modelRoles:
  slow:
    - anthropic/claude-sonnet-4-20250514
    - openai/gpt-4o
    - ollama/llama3.1

  default:
    - openai/gpt-4o
    - anthropic/claude-sonnet-4-20250514
    - ollama/llama3.1
```

Quando o modelo principal da role `slow` falha, o OMP automaticamente tenta o próximo da lista. É como ter um navio com motor principal e dois reservas — se um quebra, o próximo liga sem intervenção manual.

Path-Scoped Models

Outro knob de roteamento poderoso é o path-scoped model — definir modelos diferentes dependendo de onde o agente está trabalhando. Por exemplo, código Python pode usar um modelo especializado em Python, enquanto documentação usa um modelo barato.

```
modelRouting:
  - path: "**/*.py"
    model: anthropic/claude-sonnet-4-20250514

  - path: "**/*.md"
    model: openai/gpt-4o-mini
```

```
- path: "**/*.yaml"
  model: ollama/qwen2.5-coder
```

É como ter equipamentos diferentes no estaleiro para cada tipo de material — cortadores de aço para o casco, soldadores especiais para o motor, pintores refinados para a cabine.

Round-Robin de Credenciais

Se você tem múltiplas chaves de API do mesmo provider, o OMP suporta round-robin — rotação automática entre as chaves.

```
providers:
  openai:
    apiKey:
      - "<chave-conta-1>"
      - "<chave-conta-2>"
      - "<chave-conta-3>"
    api: openai-completions
    models:
      - id: gpt-4o
```

O OMP distribui as requisições entre as três chaves ciclicamente. Se uma chave atinge o rate limit, as outras duas continuam disponíveis.

Os 4 Knobs de Roteamento

Knob	O que faz	Quando usar
Model Roles	Mapeia funções para modelos	Sempre — é a configuração base
Fallback Chains	Lista ordenada de alternativas	Quando depende de APIs externas
Path-Scoped Models	Modelos diferentes por tipo de arquivo	Quando projetos misturam linguagens
Round-Robin	Rotação de chaves de API	Quando tem múltiplas contas

Cena de contraste: o navio que ficou parado

Imagine que você configurou seu OMP com um único provider — o Claude Sonnet 4 da Anthropic. Tudo funciona perfeitamente até segunda-feira às 9h, quando toda a equipe começa a usar o mesmo modelo para code review. O rate limit é atingido em minutos.

Seu OMP começa a retornar erros 429. O agente congela. Você fica sem resposta por 20 minutos — o tempo necessário para o rate limit resetar. O navio ficou parado no meio do oceano, sem motor reserva.

Agora veja como seria com fallback chain configurada. Quando o Claude atinge rate limit, o OMP detecta o erro automaticamente e muda para o GPT-4o em milissegundos. Se o OpenAI também estiver sobrecarregado, o agente cai no Ollama local — sem custo, sem rate limit, sem downtime. O navio nunca para de navegar.

Armadilhas comuns

Esquecer de configurar fallback para roles críticos. Se só o `default` tem fallback e o `slow` não, uma falha no modelo de raciocínio profundo trava todas as tarefas complexas.

Usar o mesmo modelo para todos os roles. Se `smol` e `slow` apontam para o mesmo modelo, você desperdiça dinheiro — tarefas baratas rodam no modelo caro.

Não testar a fallback chain. Configure, teste, quebre de propósito. Desative o provider principal e veja se o backup funciona. É como testar o motor reserva antes de zarpar.

Ignorar o Ollama para tarefas leves. Modelos locais de 7-8B são perfeitos para `tiny` e `smol` — tarefas de validação, formatação e lint não precisam de modelos frontier.

Deixar chaves de API no models.yml sem variável de ambiente. Em produção, use `${OPENAI_API_KEY}` em vez de colocar a chave diretamente no arquivo.

Próximos Passos

Neste capítulo, você descobriu que o estaleiro do OMP pode operar com uma frota inteira de motores. Frontier APIs, coding plans e self-hosting oferecem três caminhos para acessar modelos. Custom providers em models.yml dão controle total sobre quais modelos estão disponíveis. E fallback chains, path-scoped models e round-robin são os quatro knobs de roteamento que garantem que seu agente nunca fique parado.

No próximo capítulo, você vai aprender sobre Time-Traveling Stream Rules — regras inteligentes que injetam correções mid-stream quando o modelo começa a desviar.

Acesse a documentação completa: <https://omp.sh/docs>

Siga-nos nas redes sociais para dicas, tutoriais e novidades sobre o Oh My Pi.

Time-Traveling Stream Rules: Course-Correction Inteligente

GPS que corrige a rota em tempo real

No capítulo anterior, você dominou o roteamento inteligente de modelos — fallback chains, path-scoped models e round-robin credentials que garantem que o modelo certo atenda a cada tarefa.

Mas e se o próprio modelo, mesmo sendo o certo para a tarefa, starts a gerar algo fora do script? E se, no meio de um stream de saída, você pudesse detectar o desvio e corrigir o modelo antes que ele terminasse a frase — como um timoneiro que ajusta o leme no instante em que o navio começa a sair da rota?

É exatamente isso que o OMP oferece com as Time-Traveling Stream Rules.

O Problema: Modelos que Saem do Script

Todo modelo de linguagem tem moments de desvio. Você configura o roteamento perfeito — Claude para raciocínio, Grok para código, Gemini para visão — e mesmo assim o modelo começa a gerar o que não deveria. Talvez ele ignore uma regra de estilo, comece a usar uma API deprecada, ou starts a produzir código sem tratamento de erros.

Até agora, a solução era reforçar o system prompt com mais regras — mas isso tem um custo. Cada regra adicionada ao system prompt consome tokens em cada turno, mesmo quando o modelo não precisa dela. É como manter todos os equipamentos do estaleiro ligados o tempo todo, mesmo quando só precisa de um martelo.

A Solução: Stream Rules — Regras que Dormem até Serem Necessárias

As stream rules do OMP funcionam de forma fundamentalmente diferente. Em vez de serem injetadas no system prompt (pagando token a cada turno), elas ficam dormientes — monitorando o stream de saída em tempo real. Quando o modelo gera algo que casa com o regex pattern da regra, a regra “acorda” e age.

O mecanismo tem três passos.

Regex Match: o OMP aplica os patterns de regex ao stream de tokens conforme são gerados. Não é preciso esperar o turno inteiro — o match acontece token a token.

Abort Mid-Token: quando um match é encontrado, o OMP aborta o stream imediatamente, no meio do token que está sendo gerado. O modelo não termina a frase errada.

Retry com System Reminder: o OMP injeta a stream rule como um system reminder no contexto e faz retry a partir do ponto anterior ao desvio. O modelo recebe a correção e continua de onde parou — agora na rota correta.

O custo? Zero tokens adicionais em turns normais. As regras só consomem tokens quando são ativadas — quando o modelo realmente precisa da correção.

Dois Modos de Correção

As stream rules oferecem dois modos de ação, cada um adequado a um tipo de desvio.

Abort + Retry (correção dura): o stream é abortado, a regra é injetada como system reminder, e o modelo refaz a saída. Usado quando o desvio é grave — como gerar código sem tratamento de erros ou usar uma API deprecada.

Inject-Only (lembrete suave): a regra é injetada como system reminder sem abortar o stream. O modelo continua gerando, mas agora tem a informação da regra no contexto. Usado quando o desvio é leve — como uma convenção de nomenclatura que o modelo esqueceu.

A escolha entre um e outro depende da gravidade do desvio. Abort é cirúrgico; inject é preventivo.

A metáfora do navio em oceano

Imagine que você está navegando com um navio de carga pelo oceano. O compás aponta a rota correta, mas de vez em quando uma corrente marinha forte começa a empurrar o navio para fora do curso.

Sem monitoramento, o navio só descobre o desvio quando chega ao porto errado — aí já gastou combustível, tempo e dinheiro.

Agora imagine que o navio tem um sistema de monitoramento que verifica a posição a cada instante. No momento em que a corrente começa a desviar o navio, o sistema dispara um alarme, o timoneiro ajusta o leme imediatamente, e o navio volta à rota — sem nem perceber que esteve perto de sair do curso.

As stream rules são esse sistema de monitoramento. O regex pattern é o sensor que detecta a corrente. O abort é o alarme que para o navio no meio da manobra. O system reminder é a instrução que o timoneiro recebe para corrigir o rumo.

Estrutura de uma Stream Rule

Uma stream rule é declarada na seção `streamRules` do `config.yml` do OMP. Cada regra tem quatro campos essenciais.

```
streamRules:
- name: "bloquear-codigo-sem-error-handling"
  pattern: '(?i)(try|catch|except|raise|throw)\s*\{?\s*\}'
  action: "abort"
  reminder: |
    Você começou a gerar um bloco try-catch vazio. Todo tratamento de
    erros
    DEVE ter pelo menos um log ou re-throw. Corrija o bloco antes de
    continuar.
```

name: identificador único da regra (para debug e logs). **pattern:** regex pattern aplicado ao stream de tokens. **action:** "abort" para correção dura, "inject" para lembrete suave. **reminder:** o texto do system reminder que será injetado quando a regra disparar.

Regras com Múltiplos Patterns

Para cenários mais complexos, você pode declarar múltiplos patterns em uma única regra.

```
streamRules:
- name: "proibir-imports-deprecados"
  patterns:
    - 'from\s+RPi\.GPIO\s+import'
    - 'import\s+RPi\.GPIO'
    - 'import\s+RPi\.GPIO\s+as\s+GPIO'
  action: "abort"
  reminder: |
    Você está usando RPi.GPIO, que está deprecada e não suporta o Pi 5/
    RP1.
    Use gpiozero em vez dele.
```

Quando qualquer um dos patterns casa, a regra dispara. Isso permite cobrir variações de uma mesma proibição sem duplicar regras.

Regras com Escopo de Path

Assim como os path-scoped models, as stream rules podem ser restritas a diretórios específicos.

```
streamRules:
- name: "proibir-var-global-em-modulos"
  pattern: '^(?:var|let|const)\s+\w+\s*=\s*.*(?:process\.env|require) '
  action: "inject"
  reminder: |
    Evite variáveis globais com process.env em módulos compartilhados.
    Passe a configuração como parâmetro ou use um módulo de config
    centralizado.
  paths:
    - "src/shared/**"
    - "src/utils/**"
```

A regra só é monitorada quando o modelo está editando arquivos dentro dos paths listados.

Monitoramento em Tempo Real

O OMP não precisa esperar o turno inteiro para detectar o desvio. Conforme o modelo gera tokens, o OMP acumula um buffer interno. A cada token, o buffer é testado contra todos os patterns das stream rules ativas.

Se um match é encontrado, o OMP envia um sinal de abort para o provider API. O stream é interrompido imediatamente — o modelo não gera mais tokens. O system reminder da regra é injetado no contexto da sessão. E o OMP faz retry do prompt anterior ao desvio, agora com a correção no contexto.

O tempo entre o match e o abort é menor que um token — o modelo não tem chance de completar a frase errada.

Regras Persistentes vs. Regras de Sessão

As stream rules podem ser declaradas em dois níveis.

Persistentes (config.yml): ficam ativas em todas as sessões. São ideais para regras de qualidade universal — como proibir imports deprecados ou exigir tratamento de erros.

De sessão (inline): declaradas via `/rule` durante uma sessão. São ideais para correções pontuais — como uma regra específica para o módulo que você está trabalhando agora.

```
# Criar uma regra de sessão
/rule add --name "evitar-console-log-em-prod" \
  --pattern 'console\.(log|debug|info)\(' \
  --action inject \
  --reminder "Em produção, use um logger estruturado em vez de console.log"

# Listar regras ativas
/rule list

# Remover uma regra
/rule remove "evitar-console-log-em-prod"
```

Armadilhas comuns

Regex muito amplo. Se o pattern casa com coisas que não são desvios, a regra dispara atoa e interrompe o stream desnecessário. Teste seus patterns com exemplos reais antes de ativar.

Reminder vago. Se o system reminder não explica o que o modelo deve fazer em vez da ação proibida, o modelo pode simplesmente parar de gerar — e você perde mais tokens com a hesitação do que com o desvio original.

Muitas regras ativas. Cada regra consome processamento (regex match token a token). Mais de 10-15 regras ativas simultaneamente pode introduzir latência perceptível no stream.

Próximos Passos

Você agora domina três pilares das Time-Traveling Stream Rules do OMP. Regex Match e Abort — o mecanismo que detecta desvios em tempo real e interrompe o stream antes que o erro se propague. Injeção de System Reminders sob demanda — o sistema que injeta correções apenas quando o modelo precisa delas, sem poluir o contexto em turns normais. E Sobrevivência a Compaction — a garantia de que as correções injetadas persistem mesmo quando o contexto é compactado.

No próximo capítulo, você vai descobrir como o OMP lida com memória entre sessões — como o agente lembra do que aprendeu, curta fatos relevantes e mantém o conhecimento acumulado ao longo do tempo.

Acesse a documentação completa: <https://omp.sh/docs>

Siga-nos nas redes sociais para dicas, tutoriais e novidades sobre o Oh My Pi.
